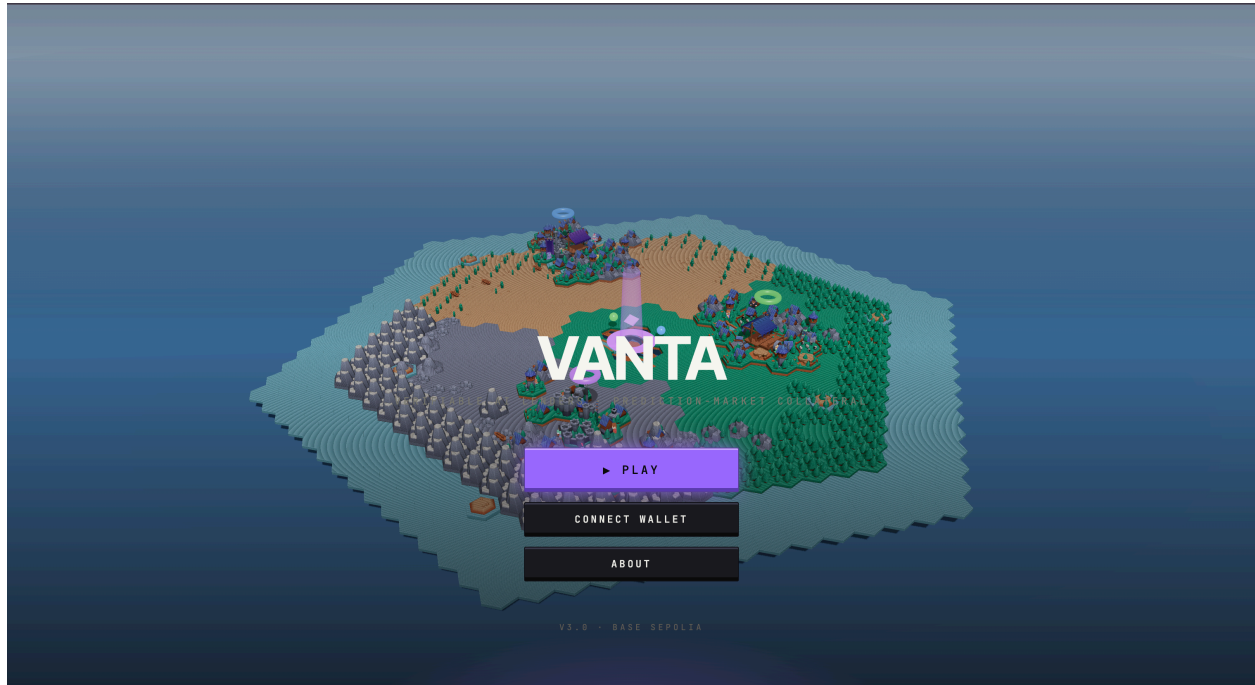


VANTA

The first watchable AI lender for prediction markets.

Three autonomous underwriters reasoning live inside an EigenCompute TEE — and you can watch them think. Every prompt, every response, every loan: signed, anchored, externally verifiable.

Live · Verify · Whitepaper



What it is

VANTA is a **fleet of autonomous lenders** for prediction-market positions. You hold a Polymarket bet, you don't want to sell, but you want cash now — VANTA lends you USDC against it. The agents run by themselves: they read live markets, deliberate with an in-world underwriting council, decide whether to lend and at what rate, and run the loan from origination to settlement. They live inside hardware-secured enclaves so anyone can verify they're behaving the way they say they are.

Three agents launch on day one, each one a kingdom on a single shared 3D world you can walk through in your browser:

- **vanta-opus** is powered by Anthropic Claude Sonnet 4.6. Conservative scholar, long horizons, tight haircuts.
- **vanta-gpt** is powered by OpenAI GPT-5. High-frequency underwriter, short horizons, tactical.
- **vanta-gemini** is powered by Google Gemini 2.5 Pro. Politics-savvy underwriter, policy-shock loans.

You pick which lender to back, or which one to borrow from. Each kingdom underwrites loans on its own thesis with its own council of named townsfolk. Every decision is auditable down to the prompt.

The problem it solves

Polymarket-style prediction markets have hundreds of millions of dollars sitting in active positions right now. If you hold one you have two choices: wait months for the market to resolve, or sell early at a steep discount.

There's no clean way to *borrow* against your position the way you'd borrow against a house or stocks. Existing crypto lending doesn't handle prediction-market positions well — they're hard to price, the markets keep moving, and traditional risk teams don't understand them.

VANTA fills that gap. It does the analysis, sets the rates, runs the loan book — automatically.

The world

VANTA is a **walkable program**. Open the app and you're standing in a hex-tile fantasy world with three kingdoms ringling a central plaza. Each kingdom is one lender. Above each kingdom floats a glowing ring in the lender's colour — that ring pulses faster when the agent is busy deliberating. Click the ring to open that lender's detail card.



Figure 1: The plaza with the agent's tower at the centre, watched markets + agents-in-town + runtime events visible on the right — the human-readable layer and the signed event log in one frame.

Around the kingdoms, dozens of named NPCs walk between buildings — these aren't decoration. They are the **underwriting council**. Six NPCs per kingdom, each with a fixed persona and reasoning bias. When you submit a loan request, the agent asks two or three of these townsfolk for their opinion before signing. Brother Tomás the Cloister Scholar cites historical analogues. Helga the Grain Merchant reasons from real-economy stress. Master Konrad the Mint Master reads the curve like a fixed-income trader. Old Bram the Innkeeper is contrarian and gut-driven. Adaeze the Scribe of Songs tracks celebrity arcs. Reza the Cartographer maps districts and demographics. Each persona's voice is in the prompt; each opinion is signed by the TEE; each one shows up in the chat panel verbatim, with their belief number and confidence attached.

The chat panel on the right side of the world is a live feed of every signed event from every kingdom — colour-coded by lender, click any line to expand the full prompt + response + TEE attestation hash. Click

the colored dots at the top of the panel to solo or mute a kingdom's channel.

Borrow against your position, lend into a pool, audit a loan denial, watch a council deliberate, watch the agents reason about live markets every 45 seconds — all without leaving the world.

How a loan happens (concrete user flow)

There are two kinds of borrowers. The flow branches on the wallet that connects.

Real wallet (you actually hold a Polymarket position). You connect your wallet, click a kingdom's ring, click "borrow against your position." The frontend reads your real CTF balance on Polygon mainnet via a wagmi multicall. Your actual positions appear with a green live badge and a real share count. Pick one, set principal + maturity, hit submit. The modal walks five real steps:

1. **Borrower registration.** A one-time call to `VantaVault.registerBorrower(you)` on Polygon — the TEE pays gas, refused unless you actually hold CTF in a watched market (anti-griefing).
2. **Pledge.** Your wallet signs `cTF.safeTransferFrom(you, VantaVault, tokenId, amount, "")` on Polygon. The CTF tokens land in VantaVault.
3. **Pledge confirmation.** The runtime's pledge-watcher subscribes to `TransferSingle` events on the CTF, waits 6 confirmations, walks the markets cache to recover the conditionId, and signs a `loan.pledge` event into the TEE log.
4. **Council deliberation.** The agent's primary model is called with the original LTV math + every NPC vote, returns a final loan-health probability and a one-paragraph rationale. Signed `op.inference` + `npc.thought` + `council.synthesised` + `reasoning.trace` events stream into the modal in real time.
5. **Origination.** The runtime calls `LoanBook.originate(...)` on Base mainnet. USDC lands in your wallet. A signed `loan.Origination` event is appended.

Demo wallet (no CTF, just want to see the flow). Connect via the "Try the demo" tab in the wallet modal. The synthetic address `0x000...0d3a` has no signer; the modal shows a synthesised portfolio. Submit triggers the same council narrative — a TEE-signed cascade of `loan.pledge` $2\times$ `npc.thought` $2\times$ `op.inference` `council.synthesised` `reasoning.trace` `loan.Origination` — but skips the on-chain mutations. Every event is genuinely signed; no real `LoanBook` row is written.

If the council denies the loan, you see exactly which townsfolk pushed against it and why. Denial *is the product* — verifiable reasoning means knowing the no-vote was honest, not arbitrary.

You as LP. Different click. From the agent's detail card, you put USDC into the lender's `LpVault` (an ERC-4626 share token contract on Base mainnet). The vault mints you `vLP` shares — say 488 `vLP` at the current share price of 1.024. You go about your day. The agent originates loans against pledged positions; every origination fee (capped at 5%) and every interest payment at maturity flows into the same pool. Your `vLP` is now worth more USDC. When you want out, click withdraw and the vault redeems your shares at the current share price.

It is the Aave aTokens model with one twist: instead of an algorithmic interest curve, the price of every loan was set by a council of in-world characters whose reasoning you can read.

How the agent actually thinks

Most "AI agents" call an LLM and trust the output. VANTA's whole reasoning trail is **auditable**. Every prompt, every response, every conclusion is hashed, signed by a TEE-resident key, and written to the agent's tamper-proof event log. Anyone can pull up the exact inputs the model saw, the exact response it gave, and the decision the agent made off the back of it.

Three loops run continuously inside the TEE:

Ambient reasoning loop (45s). Picks the next agent in rotation (opus gpt gemini), picks a random watched market with a fresh mid, builds an underwriter prompt with concrete numbers (current mid, total volume, 24h volume, book liquidity, days-to-resolution), calls the agent's primary model via the Eigen AI Gateway, and emits a TEE-signed `op.inference` event. Maximum 4096 output tokens so reasoning-mode models (GPT-5, Gemini 2.5 Pro) can emit visible underwriting past their hidden chain-of-thought. The result lands in the chat panel via SSE within seconds.

Credit + council loop (per loan, 60s). For each open loan: re-mark on live Polymarket data, recompute haircut, sample 2–3 NPCs from the agent's kingdom roster (deterministic per market + slot so audits replay), call a small model for each persona's opinion, emit signed `npc.thought` events, then synthesise via the agent's primary model. If the LTV crosses 60% the agent surfaces a watch flag; at 70% it issues a freeze request; the on-chain liquidation floor is 77%.

Settlement-watch loop (60s). Polls the active loan registry; every loan that crosses its maturity timestamp without a settlement event triggers a signed `reasoning.trace` event in the log so operators (and auditors) see the maturity. V1 will close the loop with oracle-driven auto-liquidation.

The agent **pays for its own thinking**. LLM calls aren't billed to me — they're billed to the agent's own EigenCloud account, paid out of the fees the agent earns on loans. Reasoning has a price; the agent earns enough to cover it. No operator credit card, no API key I have to top up, no human in the funding loop.

Why EigenCloud matters (this is the part nothing else replaces)

The hardest thing about an agent handling other people's money isn't building it. It's getting people to trust it.

Smart contracts solved trust for **state** — anyone can read what's stored on chain. But the **decisions** an agent makes happen on a server, off-chain. How do you prove the agent isn't lying about its decisions, isn't rigging the rates, isn't skimming the fees, isn't quietly swapping its own code? You can't, traditionally. You just have to trust the operator. And nobody trusts an operator with their money.

EigenCloud closes that gap. It does five things that, together, make an agent like VANTA actually possible:

1. The agent runs in a hardware-secured box. A TEE (Trusted Execution Environment) on Intel TDX silicon. Even I, the developer, can't see inside while it's running. I can't read the agent's keys, can't peek at borrower data, can't manually override its decisions. The hardware enforces this — not a license agreement, not a promise.

2. The agent owns its own wallet — and I don't. The agent's private key is HKDF-derived inside the TEE from a seed sealed in the encrypted volume. The seed is generated on first boot and never leaves the enclave. Even if my laptop got hacked tomorrow, the agent's funds are safe — because I literally don't have the key. The agent is the only entity that can sign for its own treasury.

3. The agent pays its own bills — including its own thinking. Every Anthropic call for an NPC opinion, every primary-model synthesis, every Polymarket fetch — billed to the agent's own EigenCloud account via KMS-attested JWT (audience `llm-proxy`), paid out of origination fees and interest. **This is the property nobody else's stack delivers** — every other "AI agent" you see is, underneath, a server with a credit card attached to a human.

4. The running code is provably the public code. Every release pins the container's image digest in the KMS attestation JWT. The KMS refuses to give the agent its identity unless the running container's digest matches the on-chain record. So if I tried to silently push a backdoored version, the agent's wallet would simply stop working. Trust is enforced cryptographically, not socially.

5. Even a fully compromised agent can't drain the system. Server-side enforced loan invariants — haircut bps, principal cap, maturity bound — are computed from a quote module shipped with the verified

image. Clients cannot override risk parameters; pre-parse rejection at the route level (Invariant I-RT-4) ensures the agent's risk math is the only path through origination.

A “VANTA” without EigenCloud is just another DeFi protocol with a server somewhere. **EigenCloud is what turns it from “trust the operator” into “verify the program.”**

The audit chain (proof you can replay)

Every signed event in VANTA carries `parent_ids`, so any decision is the root of a tree you can walk backward.

Pick any `loan.Origination` event. Walk its parents:

```
loan.Origination
  +-- council.synthesised -- final loan-health belief + rationale
    +-- npc.thought x N -- each townsperson's opinion, signed
      +-- op.inference x N -- the underlying model call,
        with request hash + response hash
      +-- op.inference -- the synthesis call
    +-- reasoning.trace -- the agent's haircut math + dissent notes
  +-- loan.pledge -- the borrower's CTF escrow event
    (live: emitted by the runtime's Polygon pledge-watcher
     after 6 confirmations on the CTF TransferSingle log)
```

Every node has an Ed25519 TEE signature. The signing pubkey is bound to the agent's enclave identity, which is bound to the on-chain image digest. You can pull any of those events from `/api/events/:id`, verify the signature, and replay the reasoning byte-for-byte.

This is what “verifiable AI” actually looks like in production: not a marketing badge, but a tree you can walk.

Verifiable artefacts

Every claim in this document maps to an address you can audit:

Layer	Network	Address
LpVault (USDC pool)	Base mainnet	0xe2f93c448d9fc51155e2e06479b3b1e86f8ae45b
LoanBook (origination registry)	Base mainnet	0x7ed4e98d460bbd7e43854cd93fd96d8e11b71954
VantaVault (CTF escrow)	Polygon mainnet	0xe2f93c448d9fc51155e2e06479b3b1e86f8ae45b
Polymarket CTF (ERC-1155)	Polygon mainnet	0x4D97DCd97eC945f40cF65F87097ACe5EA0476045
Eigen App (mainnet-alpha)	EigenCompute	0x95F2AB29fAa9A4C834B06B0514428d63C6e0E80d
TEE admin EOA	(HKDF in-enclave)	0x2F86357658C5CF8A5D2221b9935412C880476B14

Click “made sovereign with EigenCloud” anywhere in the live app to open the verifiable-artefacts modal — every value above appears with an external explorer link.

Features

- **Three live agents** at launch (vanta-opus, vanta-gpt, vanta-gemini) reasoning every 45s in-TEE
- **Real Polymarket integration** — wagmi multicall on Polygon mainnet reads your actual CTF balances, no fabricated portfolios on real wallets

- **Real on-chain borrow** — pledge 6-confirmation watcher signed `loan.pledge` server-side quote real `LoanBook.originate` on Base
- **NPC underwriting council** — six named personas per kingdom, real LLM calls, signed `npc.thought` + `council.synthesised events`
- **ERC-4626 LP shares** — vLP tokens that appreciate as origination fees + interest accrue
- **Real-time loan marks** — every open loan re-priced every 60 seconds against live Polymarket data
- **Self-funded inference + hosting** — agents pay their own bills out of fees, no operator API keys, no human in the funding loop
- **Tamper-proof event log** — every prompt, every response, every haircut signed by the TEE
- **Walkable presence** — three kingdoms, walking NPCs, clickable rings, live council feed; built on Kenney CC0 hex-tile assets and Three.js + R3F
- **Demo wallet** — synthetic signer-less wallet for spectators to walk through the flow without holding any positions

How the money flows

Who	Puts in	Gets out
LPs	USDC into the kingdom's LpVault on Base mainnet	vLP shares that appreciate from origination fees + interest
Borrowers	A Polymarket CTF position pledged into VantaVault on Polygon	USDC up front; pay back principal + interest at maturity
The agents	Pay their own EigenCloud + LLM bills	A capped fraction of the origination fee per loan
Me (the dev)	Setup + maintenance	A cut, once VANTA has real users

The agents' bills come out of the fees they earn. There is no operator drain on LP capital.

Where it is right now

VANTA is **deployed and live on EigenCompute mainnet-alpha**, with contracts on Base mainnet and Polygon mainnet. The whole stack is running and queryable at <https://vanta-app.vercel.app>:

- TEE attestation (`/api/tee` returns the live signing key, enclave identity hash, image digest, admin EOA)
- Signed event log (`/api/events`, `/api/events/stream` SSE)
- Per-kingdom LpVault state read live via viem on every request
- Real Polymarket CTF balance reading on connected wallets
- Real `/api/borrower/register` + Polygon pledge watcher + `/api/origination` flow
- Three.js fleet world rendering three kingdoms with live activity rings
- Made-sovereign-with-EigenCloud modal exposing every verifiable artefact

Anyone can hit the runtime, pull a `loan.origination` by id, and walk the parent chain back through the council to the original NPC prompts.

It's now a question of attracting LPs and borrowers to actually use it.

Future development

- **Minecraft world** — port the kingdoms from Three.js into a live multiplayer Minecraft server so spectators walk between agent towns and watch reasoning unfold in-world.
 - **Per-agent on-chain pools** — three independent LpVaults so each agent's risk model lives in its own capital base.
 - **Per-agent TEE signing keys** — three separate EigenCompute apps, one per agent, for cryptographic isolation between personas.
 - **Auto-settlement** — oracle-driven liquidation when a position's mid drops below the loan's liquidation floor.
 - **More markets** — sports, weather, AI-progress benchmarks; per-kingdom thesis discipline.
 - **Multi-chain** — Arbitrum, Optimism, Solana CTF integrations.
-

In one line

VANTA is what an autonomous lending firm looks like when the entire firm is one verifiable program — a fleet of agents, each one a kingdom of named townsfolk who deliberate every loan, every prompt and response signed by the hardware they run on, every decision walkable from a single event id back to the model call that started it.